

内部排序

在内存中怎样用局部进展制造全局有序

408 · 数据结构 Topic DS-11

母问题：每一步究竟把什么“已排序”变成事实？

内部排序的生成链是

Local Operation → Progress Measure → Ordering Invariant → Termination → Cost/Stability.

插入排序维护已排序前缀，选择排序维护已确定前缀，交换排序用 partition 缩小未决区间，归并排序把有序子段合并，堆排序借用 DS05 的根极值。算法区别不只在口诀，而在每轮不变量和数据搬运方式。

本册定位与 Owner 边界

- **Owns:** 直接/折半插入、Shell、冒泡、简单选择、快速、堆、二路归并、基数排序，以及比较/移动、稳定性、原地性与最坏输入。
- **Uses:** DS05 堆的接口与 Atlas Foundation 成本语言。
- **Stops:** 外部存储块 I/O 与多路归并归 DS12；选择哪种排序的 workload 规则进入 Rules。

目录

1	排序契约与三种性质	3
2	不变量驱动的四条主线	3
3	插入族：定位成本与移动成本必须分开	3
4	交换与选择族：每趟提交的事实不同	4
5	堆、二路归并与基数：改变候选组织方式	4
5.1	比较次数、移动次数与下界	5
5.2	从中间状态反推算法	5
5.3	按约束选型，而不是背唯一答案	5
5.4	Quickselect: partition 后只保留含目标秩的一侧	5
5.5	桶排序：线性期望来自分布假设	5
5.6	自然 runs 的工程组合	6
5.7	归并中的逆序对计数：排序过程同时给出统计量	6
5.8	计数、基数与负数：值域边界决定能否线性	6
6	代码与测试证据	6
7	复杂度与概念边界	9

1 排序契约与三种性质

排序先固定升序/降序、是否允许重复、是否要求稳定以及是否允许额外空间。稳定性保持相等 key 的原始相对次序；原地性描述额外空间，不等于稳定；比较排序的下界适用于只通过比较获得次序的算法，基数排序改变了信息模型。

“一趟”不是“执行一次循环”这么简单，而是算法完成一次可观察的全局推进：冒泡一趟把一个极值放到末端，简单选择一趟把一个极值放到前端，快排一次 partition 让枢轴归位，归并一趟合并一组有序段，基数排序一趟处理一位。排序过程题应先识别这个已确定区域，再判断当前数组是否可能来自该算法。

稳定性由相等 key 的处理符号决定：插入时用 $>$ 而不是 $\geq$ ，冒泡只交换严格逆序，归并相等时优先取左段，才能保持相对次序。稳定算法适合多关键字排序的后续主关键字排序，也是 LSD 基数排序正确性的前提；“原地”只描述额外空间，不能替代稳定性结论。

2 不变量驱动的四条主线

插入排序：前缀有序，取下一个元素向左移动直到放置点；最好 $O(n)$ 、最坏 $O(n^2)$ ，稳定。选择排序：前缀是全局最小的若干元素，交换次数少但通常不稳定。快速排序：partition 后枢轴落在最终位置，递归处理两侧；平均 $O(n \log n)$ 、最坏 $O(n^2)$ 。归并排序：两个有序段按双指针合并，稳定且 $O(n \log n)$ ，需要线性辅助空间。

为什么“有序区”不是一句空话

对数组 $[4, 1, 3, 2]$ 做插入排序。处理完 4, 1 后前缀成为 $[1, 4]$ ；处理 3 时只需在该前缀内找插入点。若每轮都重新排序整个数组，就失去该不变量带来的结构化进展。

3 插入族：定位成本与移动成本必须分开

直接插入从有序前缀末尾向左比较并移动，直到找到插入位置。折半插入只用二分缩短“找位置”的比较次数，但连续数组仍要移动插入点后的元素，所以总移动量和最坏时间仍为 $O(n^2)$ ；它不是把插入排序整体变成 $O(n \log n)$ 。

若数组下标从 1 开始，可把待插入元素暂存到 $A[0]$ 作为哨兵：向左移动时只比较 $A[j] > A[0]$ ，不必每轮再检查 $j \geq 1$ ；但哨兵只省边界判断，不改变移动次数的量级，失败/空表仍需单独处理。插入排序的后移总次数等于初始逆序对数，已排序输入没有后移，逆序输入后移达到 $n(n-1)/2$ 。

链式存储上的插入排序不需要搬移整段记录，只需摘下当前节点并按序插入已排序链；定位仍可能扫描 $O(n)$ ，总时间平均/最坏为 $O(n^2)$ ，额外空间可为 $O(1)$ 。因此“归并可自然用于链表”与“折半插入要求随机访问”是表示层差异，不是排序口诀差异。

Shell 排序选择递减增量 $d_1 > d_2 > \dots > 1$ ，每一趟对所有相距 d 的子序列做插入排序。大增量先让元素跨远距离移动，最后 $d = 1$ 时数组已经较接近有序。其正确性最终由最后一趟普通插入保证；复杂度依赖增量序列，不能在未给序列时统一宣称精确阶。跨距移动可能改变相等 key 的相对次序，因此 Shell 通常不稳定。

每趟 Shell 排序结束后，数组满足“ d -有序”：任取相距 d 的位置，前者不大于后者（等价于每个同余类子序列有序）。若下一趟增量整除上一趟，较小增量会继承一部分局部秩序；但最终只有 $d = 1$ 才能推出全局有序。不同增量序列的最好/最坏界不同，题目没有给序列时只报告依赖序列的复杂度。

折半插入只把“定位插入点”的比较降到每项 $O(\log n)$ ；连续表仍需搬移插入点后的元素，因此最

坏移动和总时间仍为 $O(n^2)$ 。它的比较复杂度不能被误写成直接插入的最好 $O(n)$ ，也不能因比较少就声称整体 $O(n \log n)$ 。

4 交换与选择族：每趟提交的事实不同

冒泡排序比较相邻逆序对并交换；一趟从左到右可把当前最大元素提交到未排序区末端。若整趟没有交换即可提前停止，因此已排序输入最好为 $O(n)$ 。简单选择排序每趟在未排序区找最小元素，与边界位置交换；它只进行 $O(n)$ 次交换，但一次跨越交换可能越过相等 key，通常不稳定。

冒泡还可记录本趟最后一次交换的位置，把下一趟的边界直接缩到该位置；交换发生在更靠前处时，后缀已经有序且不会再改变。这个优化改善常数和近有序输入表现，但最坏仍为 $O(n^2)$ 。每次相邻交换恰好消除一个逆序对，因此不带跳跃交换的冒泡总交换次数等于初始逆序对数；选择排序的远距离交换则不能套用这个等式。

简单选择的比较次数固定为 $n(n-1)/2$ ，但交换次数最多约 $n-1$ 次；若强行稳定，可把最小元素后移/插入到边界并整体右移中间段，交换次数减少的代价是移动量可能退化到 $O(n^2)$ 。稳定性不是“选择排序永远不可能”，而是当前实现的操作合同决定的。

快速排序用 partition 把区间分成枢轴两侧，并让至少一个分割边界成为进展。正确性依赖 partition 循环不变量；复杂度依赖分割是否均衡。已排序、逆序或大量重复 key 在某些固定枢轴实现中会持续产生极端分割，使递归深度和时间退化到 $O(n^2)$ 。

常见 partition 有两类语义：左右指针交换版结束时枢轴可能落在边界位置，双向扫描必须分别维护“左侧不大于、右侧不小于”的区间；Lomuto 版维护一个小于枢轴的前缀，扫描指针越过元素后再交换。两者返回的分界下标不同，递归区间必须严格排除已经归位的枢轴，否则遇到重复值可能不缩小而无限递归。随机枢轴、三数取中或三路 partition 能降低特定输入退化风险，但不改变快排不稳定的默认结论。

快排的辅助空间主要是递归栈：平衡分割时为 $O(\log n)$ ，连续极端分割时为 $O(n)$ 。尾递归消除或总是先递归较小一侧、较大一侧改用迭代栈，可以把栈空间压到 $O(\log n)$ ，但不消除最坏比较时间。链表上 partition 可以通过节点重链实现，但随机选枢轴和按下标切分都不再便宜，通常交给链式归并排序。

5 堆、二路归并与基数：改变候选组织方式

堆排序先调用 DS05 建立最大堆，每轮把根极值与未排序区末端交换，缩小堆边界并对新根下滤。已排序后缀不断扩大，时间稳定为 $O(n \log n)$ 、额外空间 $O(1)$ ，但根与末端的远距离交换使其不稳定。

二路归并把两个已有序段合成一个有序段：比较两段当前首元素，取较小者并推进对应指针；某段耗尽后复制另一段剩余部分。若相等时优先取左段，可保持稳定。自底向上的归并把长度 $1, 2, 4, \dots$ 的段逐轮合并，无论初始次序如何都为 $O(n \log n)$ ，代价是 $O(n)$ 辅助数组。

链式归并可用快慢指针切分链表，再递归合并两条有序链；合并只改 next 指针，辅助空间主要是递归栈。数组归并每一趟写回辅助数组，比较次数可能因一段提前耗尽而减少，但每个元素仍需被复制/写回一次，因此“比较少”不等于“移动为零”。

基数排序不比较完整 key，而是按某一位的桶序反复稳定分配与收集。最低位优先从低位到高位，后一趟必须稳定，才能保留前面低位已建立的次序。若有 d 位、基数为 r ，成本为 $O(d(n+r))$ ；它依赖 key 可分解为有限位，不受比较排序下界约束。

计数排序是基数排序的一个位级前置模型：当 key 落在小范围 $[0, R)$ 时，统计每个值出现次数，再用前缀和把记录稳定写回，成本为 $O(n+R)$ ，代价是值域空间。它不适合值域远大于记录数的输入，不

能把“线性”脱离 R 的前提。

计数排序的稳定写回通常从右向左扫描原数组：前缀和给出某值在输出中的最后位置，写入后递减该位置。若只输出数值而不关心记录身份，正向/逆向都能得到有序结果；若作为 LSD 基数排序的一趟，必须保持同一位相等元素的先后，否则低位已经建立的次序会被破坏。MSD 基数排序则递归处理高位桶，稳定性和额外空间取决于桶内分配策略，不能把 LSD 的稳定性结论直接套用。

5.1 比较次数、移动次数与下界

简单选择的比较次数固定为 $n(n-1)/2$ ，不随初始序列变化，但交换次数至多 $O(n)$ ；直接插入的移动量等于逆序对数量，冒泡若用三赋值交换，移动的常数通常更大。归并的比较次数受数据影响，而合并写回的移动规模是 $O(n \log n)$ 。把比较、移动和递归栈分开统计，才能解释“同为 $O(n^2)$ 但实际速度不同”。

任何只通过关键字比较的排序算法，在最坏情况下至少需要 $\Omega(n \log n)$ 次比较：判定树要覆盖 $n!$ 种输入排列，高度 h 满足 $2^h \geq n!$ 。归并和堆排序达到这个比较下界；基数、计数排序不受该证明约束，因为它们还读取 key 的位或值域。

5.2 从中间状态反推算法

看到某一趟后的序列，先找不可逆的结构证据：有序前缀可能来自插入或选择，有序后缀并且每轮固定末端极值可能来自冒泡/堆排序，枢轴两侧分别满足大小关系指向快速排序，若整个序列由长度 2^t 左右的有序段拼成则指向归并，所有元素按某一位成桶且保持桶内次序则指向基数。单凭“看起来更有序”不能唯一确定算法，必须结合题目给出的趟数、比较/交换次数或稳定性要求。

5.3 按约束选型，而不是背唯一答案

小规模且基本有序：直接插入的常数和最好情况有优势；记录很大而只想少搬移：简单选择的交换次数低；随机内存数组：快速排序平均快但需防最坏退化；要求最坏 $O(n \log n)$ 且原地：堆排序；要求稳定或适合链式归并：归并排序；key 位数小且可分解：基数/计数排序。递归栈、辅助数组、链表可改写的指针成本都应纳入同一成本向量。

5.4 Quickselect: partition 后只保留含目标秩的一侧

若只求第 k 小而不需要完整有序，执行一次 partition 后，枢轴的最终秩已经确定：目标在左侧就只处理左区间，在右侧就只处理右区间，命中则停止。随机枢轴下期望递推规模按比例缩小，期望时间 $O(n)$ ；固定枢轴连续产生极端分割时仍会退化到 $O(n^2)$ 。它与快排共享 partition 不变量，但快排递归两侧、Quickselect 只进入一侧，不能把后者的数组误认为已经排序。

重复 key 较多时，三路 partition 把区间分成“小于、等于、大于”三段；目标秩落入等值段即可直接结束，且能避免相等元素造成无效递归。若要求确定性最坏线性，需要更强的枢轴选择证明，不能只把随机枢轴的期望结论改写成最坏结论。

5.5 桶排序：线性期望来自分布假设

已知 key 落在可划分的有序范围时，可把值映射到若干互不重叠、全局有序的桶，分别排序桶内记录，再按桶序连接。若输入近似均匀、桶数与 n 同阶且桶内规模受控，期望成本可接近 $O(n)$ ；若大量记录落入同一桶，成本退化为桶内排序的最坏界。映射公式还要处理最大端点，避免最大值被映射到桶数

组之外。

桶排序与计数排序都利用值域，却不相同：计数排序为每个离散 key 计数，桶排序允许一个桶覆盖区间并继续排序。跨桶全局有序来自桶的范围互不交叠；桶内稳定性和最终稳定性由具体容器与排序方法决定，不能从“分桶”二字自动推出。

5.6 自然 runs 的工程组合

工程排序可以先识别输入中已有的升/降自然 runs，对短 run 用插入策略扩展，再按受控规则归并；其思想是利用局部已有序，组合 DS11 已证明的插入与归并机制。具体最小 run 长度、合并栈规则和临时空间属于实现版本合同，因此这里只保留机制层 Extension，不宣称某语言或运行时永远采用同一算法，也不把实现常数写成 408 的固定复杂度结论。

一张表不能替代机制

遇到排序过程题，先写本趟提交的不变量：插入是有序前缀，冒泡是末端极值，选择是前端极值，快速是分区边界，堆是根极值加堆边界，归并是有序段，基数是已处理位上的稳定次序。然后才数比较、移动、趟数和空间。

5.7 归并中的逆序对计数：排序过程同时给出统计量

归并两个有序段时，若右段当前元素 $R[j]$ 小于左段当前元素 $L[i]$ ，则左段从 i 到末尾的所有元素都大于 $R[j]$ ，一次性贡献 $|L| - i$ 个逆序对；若取左元素则不新增逆序。递归返回左右段内部的计数，再加上跨段计数，得到总逆序对。这里的“左段剩余数”成立的关键是不变量：两个输入段在合并前已各自有序。

该算法把计数和归并共享同一次扫描，时间 $O(n \log n)$ 、辅助空间 $O(n)$ ；直接枚举所有下标对为 $O(n^2)$ ，适合作为小规模对拍基线。计数可能达到 $n(n-1)/2$ ，实现应使用足够宽的整数类型。重复 key 不构成逆序，判定必须使用严格的 $>$ 而非 $\geq$ 。

5.8 计数、基数与负数：值域边界决定能否线性

计数排序可把任意整数区间 $[min, max]$ 平移到非负下标 $key - min$ ，但空间由 $R = max - min + 1$ 决定；少量记录落在巨大稀疏区间时，不能因为 key 是整数就强行开数组。基数排序处理有符号整数时，可将负数与非负数分开排序：负数按绝对值排序后逆序，再与非负结果拼接；或者把符号位映射到无符号排序域，但必须证明映射保持有符号全序。

LSD 基数每一趟必须稳定，计数排序的前缀和加稳定写回正是常用实现；MSD 基数则按高位递归分桶，不能把 LSD 的稳定性和空间结论直接套用。负数、最大端点、空输入和 R 溢出都应单独测试；若 key 不是固定宽度可分解的有限位串，应回到比较排序或明确编码方案。

6 代码与测试证据

完整实现位于 `code/ds11_internal_sort.hpp`，测试位于 `code/ds11_internal_sort_test.cpp`。

```
1
2 inline void insertion_sort(std::vector<int>& values) {
3     for (std::size_t i = 1; i < values.size(); ++i) {
4         int value = values[i]; std::size_t j = i;
```

```

5     while (j > 0 && values[j - 1] > value) { values[j] = values[j - 1]; --j; }
6     values[j] = value;
7 }
8 }
9
10 inline void selection_sort(std::vector<int>& values) {
11     for (std::size_t i = 0; i < values.size(); ++i) {
12         std::size_t minimum = i;
13         for (std::size_t j = i + 1; j < values.size(); ++j) if (values[j] < values[
14             minimum]) minimum = j;
15         std::swap(values[i], values[minimum]);
16     }
17 }
18 inline void merge(std::vector<int>& values, std::vector<int>& buffer, std::
19     size_t left, std::size_t mid, std::size_t right) {
20     std::size_t i = left, j = mid, k = left;
21     while (i < mid && j < right) buffer[k++] = values[i] <= values[j] ? values[i
22         ++] : values[j++];
23     while (i < mid) buffer[k++] = values[i++];
24     while (j < right) buffer[k++] = values[j++];
25     for (k = left; k < right; ++k) values[k] = buffer[k];
26 }
27 inline void merge_sort_impl(std::vector<int>& values, std::vector<int>& buffer,
28     std::size_t left, std::size_t right) {
29     if (right - left < 2) return;
30     const auto mid = left + (right - left) / 2;
31     merge_sort_impl(values, buffer, left, mid); merge_sort_impl(values, buffer,
32         mid, right); merge(values, buffer, left, mid, right);
33 }
34 inline void merge_sort(std::vector<int>& values) { std::vector<int> buffer(
35     values.size()); merge_sort_impl(values, buffer, 0, values.size()); }
36
37 inline std::size_t partition(std::vector<int>& values, std::size_t left, std::
38     size_t right) {
39     const int pivot = values[right - 1]; std::size_t store = left;
40     for (std::size_t i = left; i + 1 < right; ++i) if (values[i] < pivot) { std::
41         swap(values[i], values[store]); ++store; }
42     std::swap(values[store], values[right - 1]); return store;
43 }
44 inline void quick_sort_impl(std::vector<int>& values, std::size_t left, std::
45     size_t right) {
46     if (right - left < 2) return;
47     const auto pivot = partition(values, left, right); quick_sort_impl(values,
48         left, pivot); quick_sort_impl(values, pivot + 1, right);
49 }
50 inline void quick_sort(std::vector<int>& values) { quick_sort_impl(values, 0,
51     values.size()); }
52
53 inline void heap_sort(std::vector<int>& values) {
54     auto sift_down = [&values](std::size_t start, std::size_t end) {
55         std::size_t root = start;

```

```

46     while (2 * root + 1 < end) {
47         std::size_t child = 2 * root + 1;
48         if (child + 1 < end && values[child] < values[child + 1]) ++child;
49         if (values[root] >= values[child]) break;
50         std::swap(values[root], values[child]); root = child;
51     }
52 };
53 for (std::size_t start = values.size() / 2; start > 0; --start) sift_down(
    start - 1, values.size());
54 for (std::size_t end = values.size(); end > 1; --end) { std::swap(values[0],
    values[end - 1]); sift_down(0, end - 1); }
55 }
56
57 inline void radix_sort_nonnegative(std::vector<int>& values) {
58     if (std::any_of(values.begin(), values.end(), [](int value) { return value <
        0; })) throw std::invalid_argument("radix sort requires nonnegative keys");
59     int maximum = 0; for (const int value : values) maximum = std::max(maximum,
        value);
60     std::vector<int> buffer(values.size());
61     for (int place = 1; maximum / place > 0; place *= 10) {
62         std::vector<std::size_t> count(10, 0);
63         for (const int value : values) ++count[static_cast<std::size_t>((value /
        place) % 10)];
64         for (std::size_t digit = 1; digit < 10; ++digit) count[digit] += count[
        digit - 1];
65         for (std::size_t i = values.size(); i > 0; --i) { const int value = values[
        i - 1]; const std::size_t digit = static_cast<std::size_t>((value / place) %
        10); buffer[--count[digit]] = value; }
66         values.swap(buffer);
67         if (place > maximum / 10) break;
68     }
69 }
70 } // namespace ipara::ds11
71
72 #endif

```

代码清单 1: 插入、归并与快速排序核心转移

```

1     const std::vector<int> input{4, 1, 3, 2, 2};
2     for (auto sorter : {ipara::ds11::insertion_sort, ipara::ds11::selection_sort,
        ipara::ds11::merge_sort, ipara::ds11::quick_sort, ipara::ds11::heap_sort})
        {
3         auto values = input; sorter(values); assert((values == std::vector<int>{1,
        2, 2, 3, 4}));
4     }
5     std::vector<int> empty; ipara::ds11::merge_sort(empty); assert(empty.empty())
        ;
6     std::vector<int> sorted{1, 2, 3}; ipara::ds11::quick_sort(sorted); assert((
        sorted == std::vector<int>{1, 2, 3}));
7     std::vector<int> nonnegative{170, 45, 75, 90, 802, 24, 2, 66}; ipara::ds11::
        radix_sort_nonnegative(nonnegative); assert((nonnegative == std::vector<int>
        >{2, 24, 45, 66, 75, 90, 170, 802}));

```



```
8  bool negative = false; std::vector<int> with_negative{1, -1}; try { ipara::
    ds11::radix_sort_nonnegative(with_negative); } catch (const std::
    invalid_argument&) { negative = true; } assert(negative);
9 }
```

代码清单 2: 空、重复、已排序与逆序边界测试

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds11_internal_sort_test.cpp -o /tmp/ds11_test
2 /tmp/ds11_test
```

7 复杂度与概念边界

算法	最好	平均	最坏	稳定/空间
直接/折半插入	n	n^2	n^2	稳定、原地
冒泡	n	n^2	n^2	稳定、原地
简单选择	n^2	n^2	n^2	不稳定、原地
Shell	依增量	依增量	依增量	不稳定、原地
归并	$n \log n$	$n \log n$	$n \log n$	稳定、 $O(n)$
快速	$n \log n$	$n \log n$	n^2	通常不稳定
堆	$n \log n$	$n \log n$	$n \log n$	不稳定、原地
基数	$d(n + r)$	$d(n + r)$	$d(n + r)$	稳定分配、 $O(n + r)$

8 做题控制协议

先写“哪一段已排序/哪个枢轴已归位”，再数比较、移动和辅助空间。看到稳定性要求优先排除会跨越相等 key 的交换；看到最坏保证与线性空间权衡，比较归并、堆与快速的不同前提。DS05 只提供堆操作，堆排序的整体不变量由本册负责。

压缩复原

五问：已确定区域在哪里？一次局部操作扩大了什么？终止量是什么？相等 key 的次序是否保留？最坏输入会让哪一支退化？